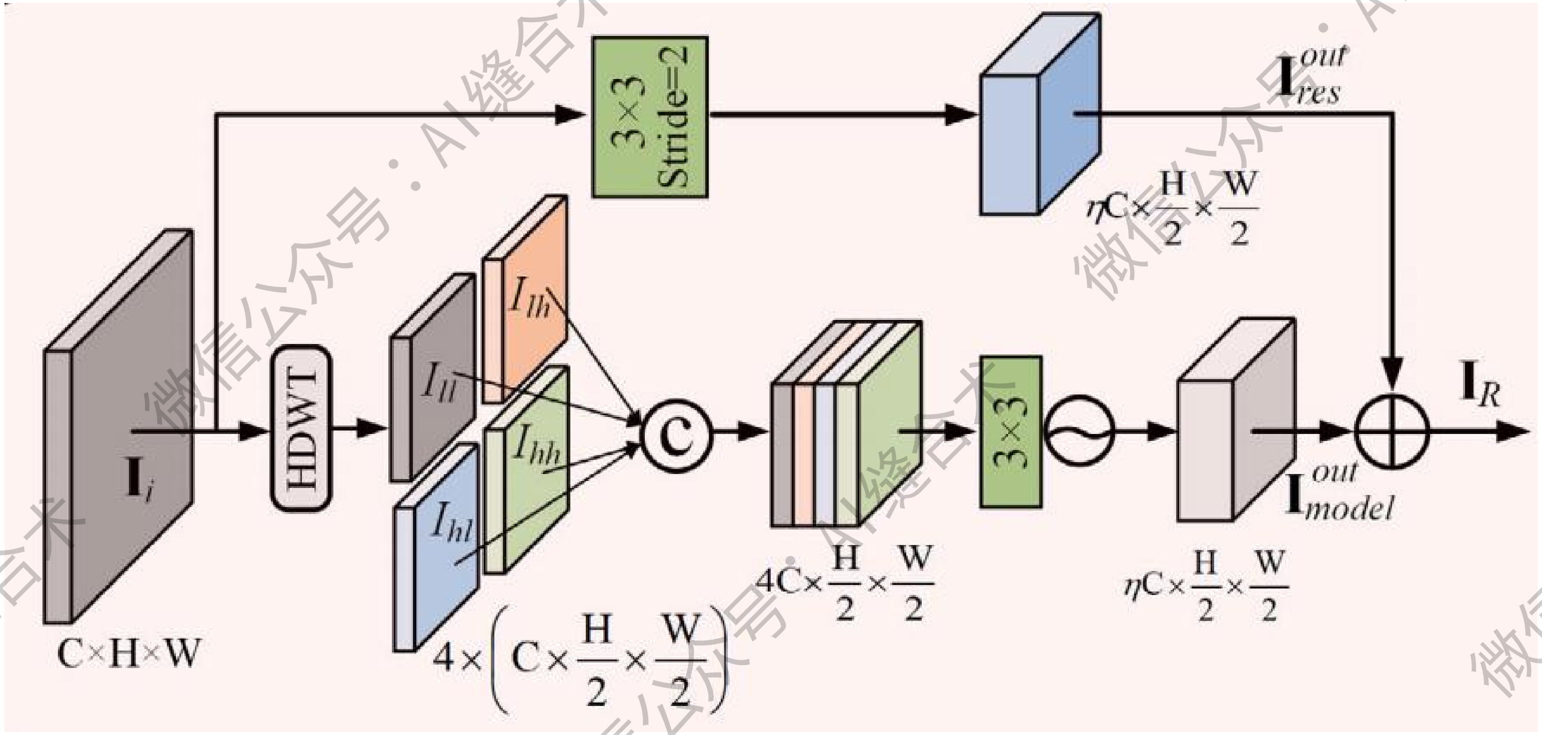
论文信息



论文题目： ASCNet: Asymmetric Sampling Correction Network for Infrared Image Destriping

中文题目： ASCNet：用于红外图像条带校正的非对称采样校正网络

论文链接： https://ieeexplore.ieee.org/document/10855453

官方 github： https://github.com/xdFai/ASCNet/tree/main

所属机构： 西安电子科技大学光电工程学院等

核心速览： 本文介绍了一种名为非对称采样校正网络（ASCNet）的红外图像条纹去除方法，该方

法能够有效捕获全局列关系并嵌入到 U 型框架中，提供全面的判别性表示和无缝的语义连接。

残差 Haar 离散小波变换（RHDWT）下采样：RHDWT 是一种新颖的下采样器，采用双分支建模有效整合条纹方向先验知识和数据驱动的语义交互，以丰富特征表示。

1. 不设置通道扩展，仅缩减特征图大小：

```
RHDWT(
  (identety): Conv2d(32, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
  (DWT): DWTForward()
  (dconv_encode): Sequential(
    (0): Conv2d(128, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (IDWT): DWTInverse()
)
输入张量的形状: torch.Size([1, 32, 256, 256])
输出张量的形状: torch.Size([1, 32, 128, 128])
```



```
RHDWT(
  (identety): Conv2d(64, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
  (DWT): DWTForward()
  (dconv_encode): Sequential(
    (0): Conv2d(256, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (IDWT): DWTInverse()
)
输入张量的形状: torch.Size([1, 64, 128, 128])
输出张量的形状: torch.Size([1, 64, 64, 64])
```

```
RHDWT(
  (identety): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
  (DWT): DWTForward()
  (dconv_encode): Sequential(
    (0): Conv2d(512, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (IDWT): DWTInverse()
)
输入张量的形状: torch.Size([1, 128, 64, 64])
输出张量的形状: torch.Size([1, 128, 32, 32])
```


2. 设置通道扩展为 2，并缩减特征图大小：

```
RHDWT(
  (identety): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
  (DWT): DWTForward()
  (dconv_encode): Sequential(
    (0): Conv2d(128, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (IDWT): DWTInverse()
)
输入张量的形状: torch.Size([1, 32, 256, 256])
输出张量的形状: torch.Size([1, 64, 128, 128])
```



```
RHDWT(
  (identety): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
  (DWT): DWTForward()
  (dconv_encode): Sequential(
    (0): Conv2d(256, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (IDWT): DWTInverse()
)
输入张量的形状: torch.Size([1, 64, 128, 128])
输出张量的形状: torch.Size([1, 128, 64, 64])
```

```
RHDWT(
  (identety): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
  (DWT): DWTForward()
  (dconv_encode): Sequential(
    (0): Conv2d(512, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): LeakyReLU(negative_slope=0.01, inplace=True)
  )
  (IDWT): DWTInverse()
)
输入张量的形状: torch.Size([1, 128, 64, 64])
输出张量的形状: torch.Size([1, 256, 32, 32])
```